

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



BITÁCORA DE RAMA: Abraham-NPCs

PROYECTO FINAL: SISTEMA ESTELAR KEPLER-AULA

NOMBRE COMPLETO: Gómez Aceves Abraham Agustín

Nº de Cuenta: 320093290

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 7 – Noviembre – 2026

CALIFICACIÓN: _____

BITÁCORA DE LA RAMA:

1.- Registro, por fecha, de lo que hice en la rama Abraham-NPCs del repositorio del proyecto: qué me propuse hacer, qué hice y cuántas horas trabajé, con las secciones de código que agregué o modifiqué y capturas de su funcionamiento.

2.- Problemas que tuve durante el desarrollo y cómo los resolví.

3.- Conclusión:

a. Complejidad y explicación de lo realizado.

b. Comentarios generales.

c. Conclusión.

4. Bibliografía en formato APA.

Datos de la rama

Repositorio: github.com/chavez-inst/Proyecto_Final_CG_Lab

Rama: Abraham-NPCs (los modelos por piezas también se integraron a main, como pidió el equipo).

Mi parte del proyecto: universo de Plants vs. Zombies: Garden Warfare. Avatar Carnívora (Chomper), NPCs de PvZ y el planeta Verdania.

Herramientas: Visual Studio (C++, OpenGL, GLFW, GLEW, GLM y Assimp), Blender para separar los modelos por piezas y GitHub Desktop para las ramas y los merges.

Periodo: del 15 al 31 de octubre de 2026, 24 commits.

Horas trabajadas en total: 21.5 horas en 5 días de trabajo (15-16, 18, 24, 27 y 31 de octubre).

Registro de commits

Fecha	Commit	Mensaje	Qué se hizo
15/10	`8e04427`	Create 1	Archivo temporal para crear la carpeta Models/Carnivora desde GitHub (después se borró).
16/10	`dc03588`	Add files via upload	Subida del modelo de la Carnívora con su LICENCIA.txt.
16/10	`fe0907e`	Add files via upload	Subida de las texturas de la Carnívora.
16/10	`e9bfc16`	Delete Models/Carnivora/1	Se borró el archivo temporal de la carpeta.
16/10	`c955d2d`	Update print statement…	Primera carga de la Carnívora en main.cpp (el título lo puso GitHub, no corresponde al cambio).
16/10	`5b570a1`	Update main.cpp	Comentario de la sección del avatar Carnívora.
16/10	`6bdc16f`	modelo de carnivora por partes	Carnívora separada en 15 piezas (.obj) para la jerarquía.
16/10	`df56992`	texturas movidas	Texturas movidas a Models/Carnivora/Textures.
18/10	`15fc3e7`	forma de carga de texturas	Model.cpp busca primero las texturas en la carpeta del propio modelo.
18/10	`6d6c256`	controles actualizados	Cámara: Espacio sube, Ctrl izquierdo baja y Shift duplica la

Fecha	Commit	Mensaje	Qué se hizo
			velocidad.
24/10	1ba409c	Create 1	Archivo temporal para crear carpetas desde GitHub.
24/10	663b97c	Subida de modelos	Los 13 modelos de PvZ por piezas (296 archivos) con licencia y piezas.txt.
24/10	67aed5b	Merge branch 'main'…	Se integró main a la rama Abraham-NPCs.
24/10	b7007a3	Delete 1	Se borró el archivo temporal.
24/10	5d6cb8d	textures	Texturas acomodadas en la carpeta Textures de cada modelo.
24/10	0326749	creando mi mundo	Verdania como zona 3: piso de pasto, skybox y cámara de llegada.
24/10	f75d4bd	animaciones implementadas	Peashooter y Crazy Dave por piezas, caminando.
27/10	bbc7e77	mi planeta	Verdania en el espacio: la cabeza del Peashooter girando.
27/10	ca7e2f7	cambios	Planeta liso con textura y las teclas Z/X recorren los 3 planetas.
27/10	8ca50e5	animaciones	Tecla para las animaciones, demás NPCs y plantas de adorno.
27/10	63aea60	mas modelos	Jardín de Crazy Dave: casa, calle, camino de piedra, cercas y plantas.
31/10	e03e91c	simplificacion de animaciones	Una sola parte animada por planta.
31/10	a7dcc9d	Merge branch 'main'…	Se integró main (cámaras, luces, día/noche) resolviendo el conflicto.
31/10	547016d	segundo personaje controlable	La tecla T cambia el avatar entre Kirby y la Carnívora.

Sección 1.

Entrada 1 (15 y 16 de octubre): avatar Carnívora separado por piezas

Commits: 8e04427, dc03588, fe0907e, e9bfc16, c955d2d, 5b570a1, 6bdc16f, df56992

Qué me propongo hacer: Descargar el modelo de la Carnívora, separarlo por piezas en Blender y cargarlo en el proyecto para ver que se viera bien.

Horas trabajadas: 3 horas

Mi parte del proyecto es el universo de Plants vs. Zombies: Garden Warfare. Lo primero fue el avatar, la planta Carnívora (Chomper), que es un personaje no humanoide.

• **Modelo descargado y con licencia.** Se descargó el modelo «Chomper - PVZ Garden Warfare» de Sketchfab (licencia CC-BY-4.0) y se guardó su crédito en Models/Carnivora/LICENCIA.txt, como pide la licencia.

• **Separación por piezas.** Para poder usarlo con jerarquía, el modelo se separó en Blender usando sus huesos en 15 piezas: base, tres tallos, cabeza, mandíbula, lengua, cuatro hojas-pata y cuatro lianas. Cada pieza es un .obj y todas comparten el mismo origen (pies en y = 0, mirando hacia +Z), así que no hay que moverlas para armar el modelo.

• **Primera carga.** En main.cpp se agregó un Model y se cargó la base para comprobar que se veía bien. Después el equipo armó los arreglos de piezas, padres y pivotes para dibujarla con DibujaJerarquia.

• **Texturas en la carpeta del modelo.** Por acuerdo del equipo, Textures/ es para las texturas del entorno, y las de cada personaje van en Models/<modelo>/Textures/. Por eso se movieron ahí.

Secciones de código que modifiqué

1) Carga del modelo en main.cpp

```
Model Carnivora_M;

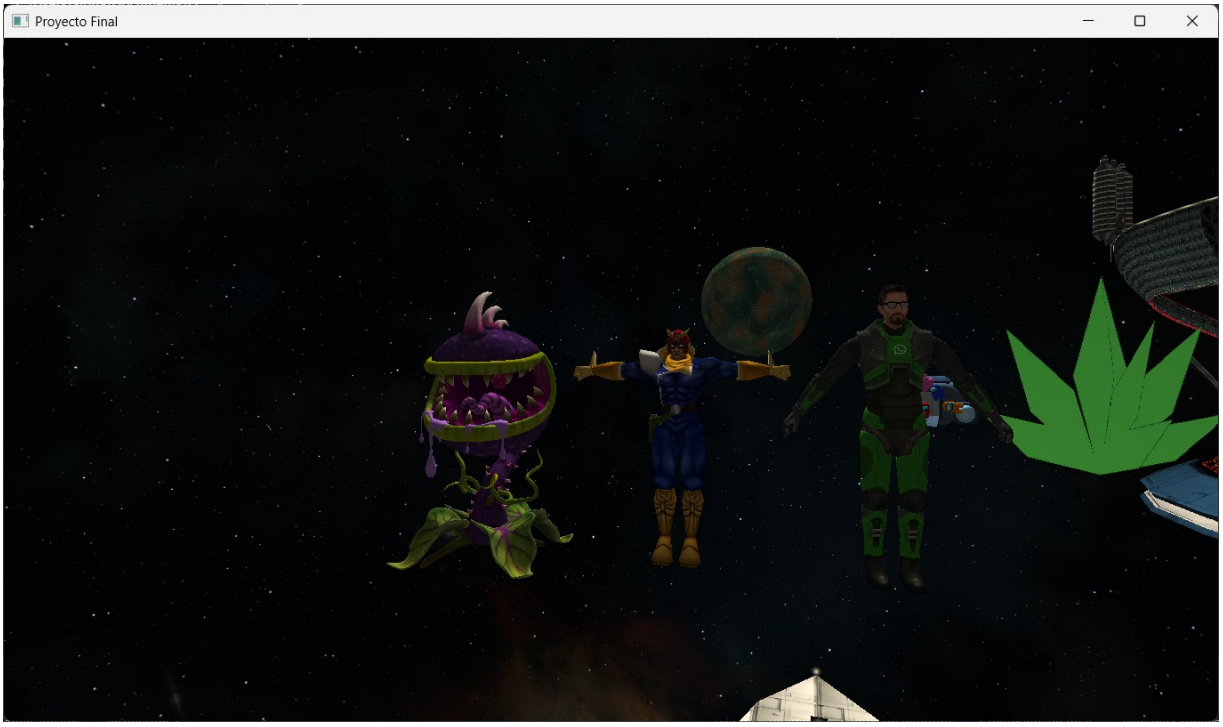
...

    Carnivora_M = Model();
    Carnivora_M.LoadModel("Models/Carnivora/carnivora_base.obj");

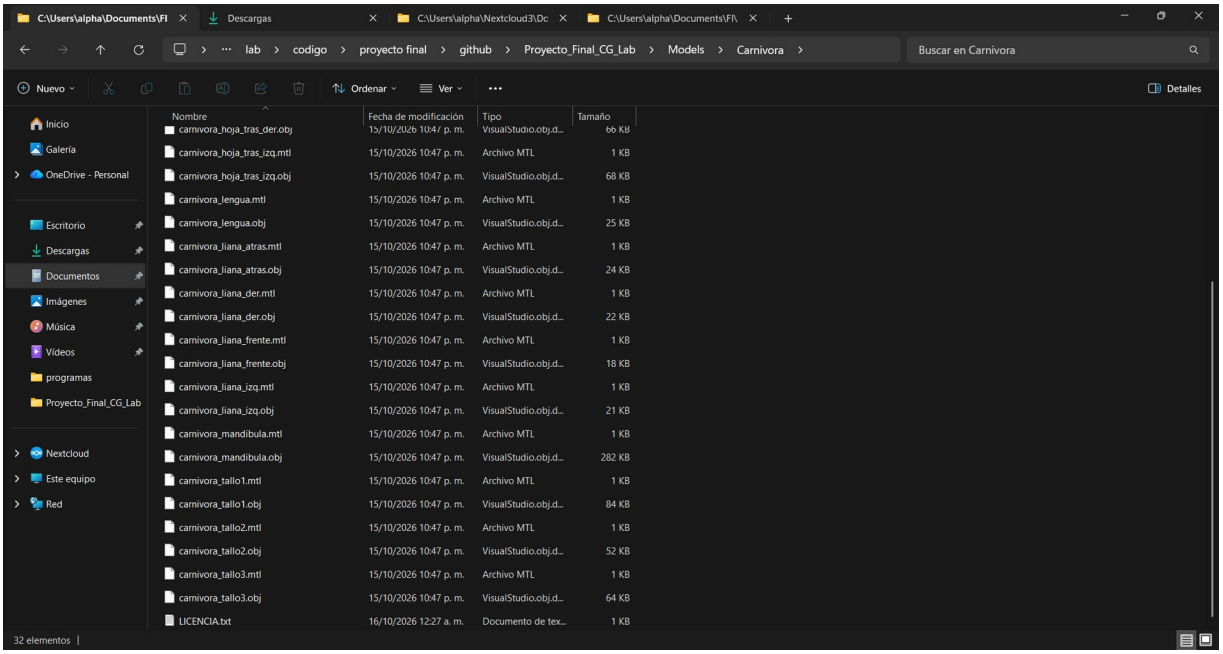
...

//modelo de avatar de carnivora
```

Capturas de pruebas de funcionamiento



Captura 1: La Carnívora en la fila de personajes del espacio (cámara libre al iniciar el programa, acercándose a la fila de personajes).



Captura 2: La carpeta Models/Carnivora en el explorador de archivos, mostrando los 15 .obj de las piezas, LICENCIA.txt y la carpeta Textures.

Entrada 2 (18 de octubre): carga de texturas por carpeta y controles de la cámara

Commits: 15fc3e7, 6d6c256

Qué me propongo hacer: Hacer que las texturas de cada personaje se carguen desde la carpeta de su propio modelo y agregar teclas para subir, bajar y correr con la cámara.

Horas trabajadas: 1.5 horas

• **Texturas por carpeta del modelo.** Model.cpp sólo buscaba las texturas en Textures/, así que las de Models/Carnivora/Textures no se encontraban y el modelo salía blanco. Se agregó a la clase Model la variable carpeta, que guarda la carpeta del .obj al cargarlo (todo lo que va antes de la última / o \). Al cargar cada textura, primero se busca en carpeta + "Textures/" y, si no existe, en Textures/ como antes. Así los modelos que ya funcionaban no cambian.

• **Controles de la cámara libre.** En Camera.cpp se agregaron tres teclas usando el mismo arreglo keys que ya usan W, A, S y D: la barra espaciadora sube, Ctrl izquierdo baja y Shift izquierdo duplica la velocidad. Para subir y bajar se usa worldUp (el eje Y del mundo) y no la dirección de la cámara, para que el movimiento siempre sea vertical aunque se esté viendo hacia abajo o hacia arriba.

Secciones de código que modifiqué

1) Model.h y Model.cpp: búsqueda de la textura en la carpeta del modelo

```
-
+#include <fstream>

...

+    //se guarda la carpeta del modelo (todo hasta el ultimo / o \, igual que en LoadMaterials)
+    carpeta = fileName.substr(0, fileName.find_last_of("\\/") + 1);

...

-                std::string texPath = std::string("Textures/") + filename;
+                //primero se busca en la carpeta Textures del propio modelo (ej. Models/Carnivora/Textures/)
+                //si ahi no esta, se busca en Textures/ como antes
+                std::string texPath = carpeta + "Textures/" + filename;
+                if (!std::ifstream(texPath).good())
+                {
+                    texPath = std::string("Textures/") + filename;
+                }

...

+    std::string carpeta; //carpeta del modelo (ej. "Models/Carnivora/"), ahi se buscan primero sus texturas
```

2) Camera.cpp: subir, bajar y correr

```
    //con Shift izquierdo presionado se mueve al doble de velocidad
    if (keys[GLFW_KEY_LEFT_SHIFT])
    {
        velocity = velocity * 2.0f;
    }

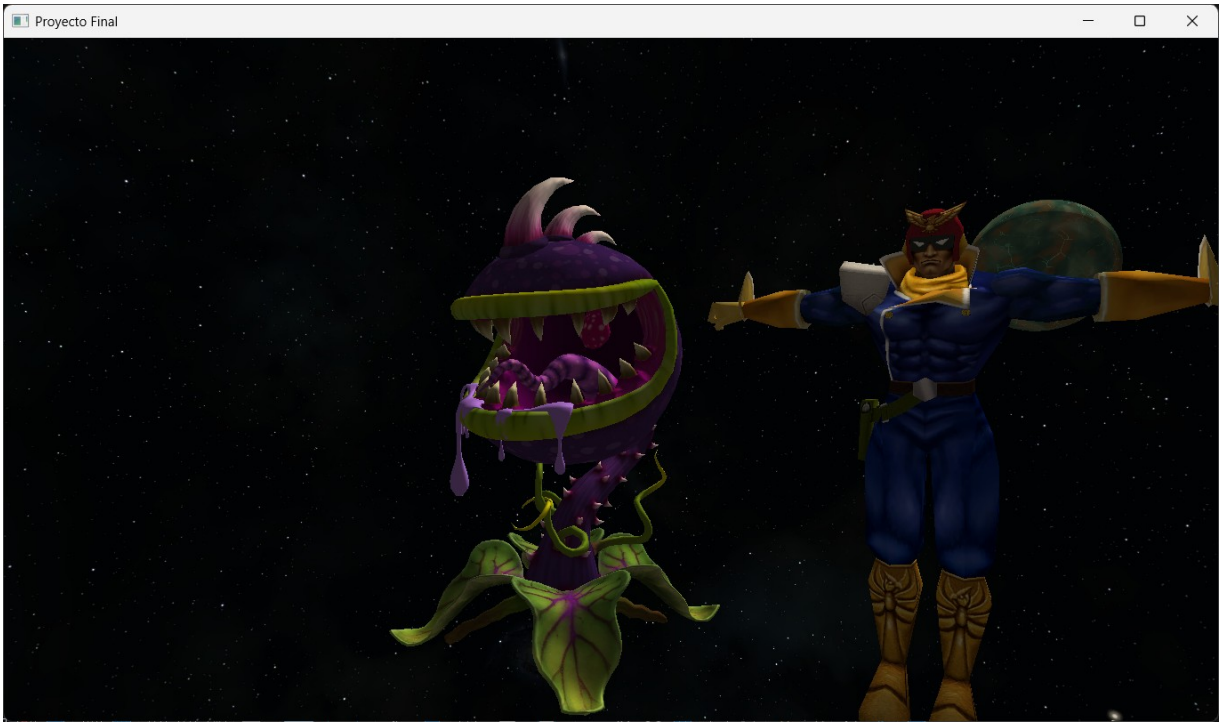
...

    //Espacio sube y Ctrl izquierdo baja, siempre en vertical (worldUp), sin importar a donde se este viendo
```

```
    if (keys[GLFW_KEY_SPACE])
    {
        position += worldUp * velocity;
    }

    if (keys[GLFW_KEY_LEFT_CONTROL])
    {
        position -= worldUp * velocity;
    }
```

Capturas de pruebas de funcionamiento



Captura 3: La Carnívora con su textura correcta (verde, con dientes y hojas de color), cargada desde Models/Carnivora/Textures.



Captura 4: Vista desde arriba de la escena después de subir con la barra espaciadora (por ejemplo, viendo la fila de personajes desde lo alto).

Entrada 3 (24 de octubre): modelos de Plants vs. Zombies por piezas

Commits: 1ba409c, 663b97c, 67aed5b, b7007a3, 5d6cb8d

Qué me propongo hacer: Separar por piezas los modelos de Plants vs. Zombies que se van a usar en Verdania y subirlos con sus texturas y licencias.

Horas trabajadas: 4 horas

Se subieron los modelos de los NPCs y de las plantas de adorno de Verdania, todos descargados de Sketchfab con licencia CC-BY-4.0 y separados en Blender por sus huesos, igual que la Carnívora.

• **NPCs:** Peashooter (12 piezas), Crazy Dave (17), Ice Pea (8), Gatling Pea (9), Fire Pea (8) y Cactus (6).

• **Adornos:** Sunflower de GW1 y de GW2, Stuffy Flower (12 piezas cada uno), Hypno-shroom (4), Laser Bean (3), Potato Mine (3), Torchwood (9) y Wall-nut (una sola pieza).

• **Archivos de cada carpeta.** Cada modelo lleva sus .obj y .mtl, su carpeta Textures, un LICENCIA.txt con el crédito del autor y un piezas.txt. El piezas.txt dice el padre y el pivote de cada pieza, que es justo lo que se copia a los arreglos padres[] y pivotes[] para dibujarlas con DibujaJerarquia.

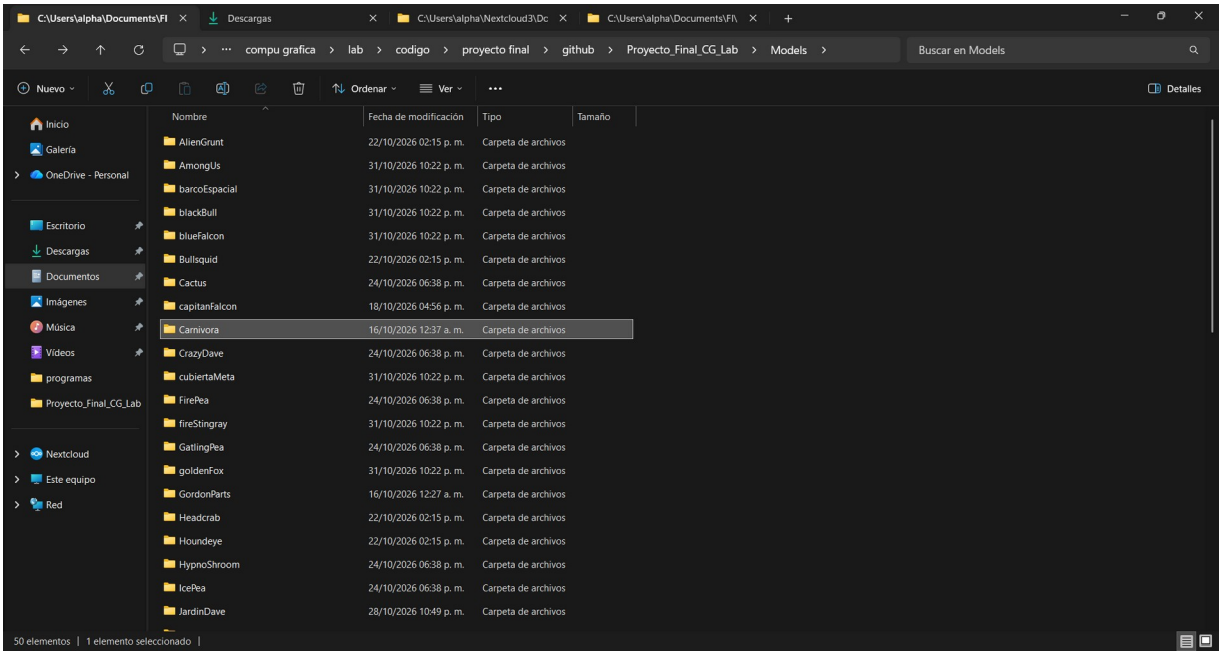
• **Rama y main.** Como pidió el equipo, los modelos por piezas con sus texturas se integraron también a main, y las animaciones y el planeta se quedaron en la rama Abraham-NPCs.

Secciones de código que modifiqué

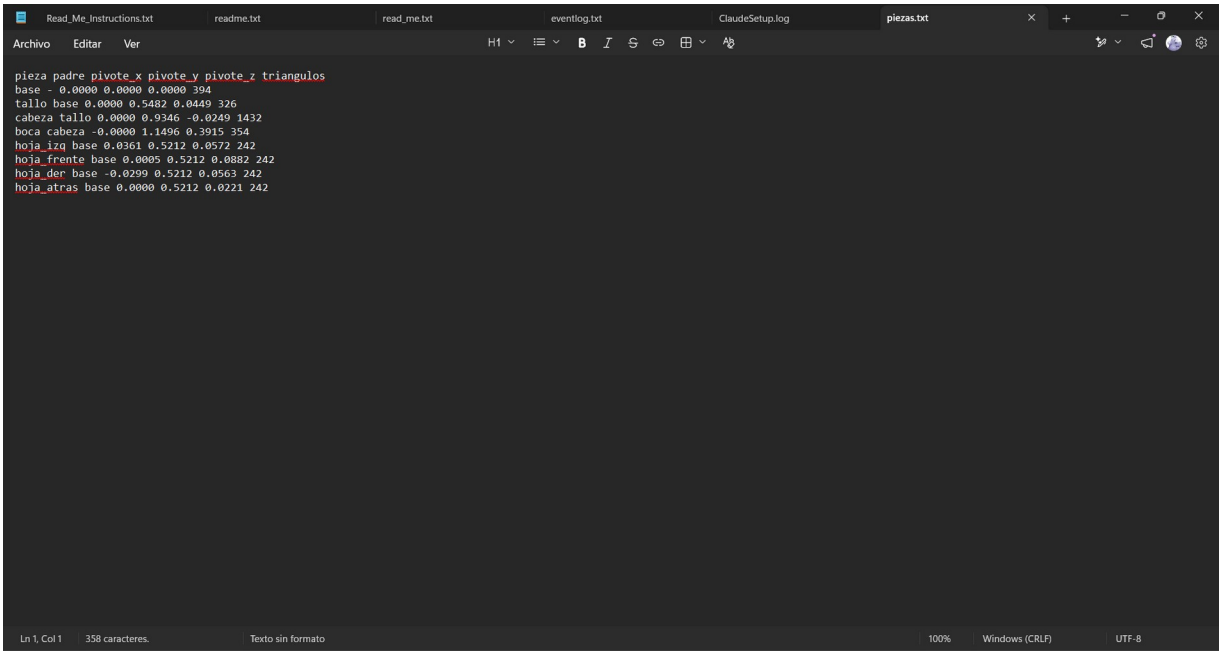
1) Ejemplo de piezas.txt (Peashooter): padre y pivote de cada pieza

```
//Models/Peashooter/piezas.txt (pivotes sacados de los huesos del modelo)
pieza padre pivote_x pivote_y pivote_z triangulos
base - 0.0000 0.0000 0.0000 971
tallo1 base 0.0000 0.2630 0.2242 664
tallo2 tallo1 0.0000 0.6249 0.1036 1104
cabeza tallo2 0.0000 0.8897 0.0169 5349
pelo_izq cabeza 0.0581 1.3543 -0.2124 523
pelo_der cabeza -0.0751 1.3665 -0.1937 520
brazo_izq tallo2 0.0583 0.6875 0.0730 1704
brazo_der tallo2 -0.0583 0.6875 0.0730 1704
hoja_del_izq base 0.0622 0.2206 0.2834 576
hoja_del_der base -0.0622 0.2206 0.2834 576
hoja_tras_izq base 0.0622 0.2206 0.1582 576
hoja_tras_der base -0.0622 0.2206 0.1582 576
```

Capturas de pruebas de funcionamiento



Captura 5: La carpeta Models en el explorador, mostrando las carpetas de todas las plantas (Peashooter, CrazyDave, IcePea, GatlingPea, etc.).



Captura 6: El archivo piezas.txt de una planta abierto en el Bloc de notas o en Visual Studio.

Entrada 4 (24 de octubre): superficie de Verdania

Commits: 0326749

Qué me propongo hacer: Crear la superficie de Verdania con su piso de pasto y su skybox, y poder llegar a ella desde el espacio.

Horas trabajadas: 1.5 horas

Mi planeta se llama Verdania. Igual que Xen y el planeta 2, su superficie es un piso aparte y lejos de lo demás, en posVerdania = (0, 0, 3000), y es la zona 3 del proyecto.

• **Piso de pasto.** Se reutilizó la malla del piso (meshListModel[0]) con la misma escala que el de Xen (200 × 200) y la textura verdania_pasto.png.

• **Skybox propio.** Se cargó el skybox de Verdania con sus 6 caras en el mismo orden que los otros (rt, lf, dn, up, bk, ft) y se dibuja cuando la zona actual es 3.

• **Cámara de llegada.** Al entrar a la zona 3, la cámara se coloca un poco atrás y arriba del centro del piso, mirando hacia las plantas.

• **Reloj de las animaciones.** Se agregó anguloVerdania, que avanza cada frame con deltaTime, igual que el ángulo de las islas de Xen. Todas las animaciones de las plantas salen del seno de este ángulo.

Secciones de código que modifiqué

1) Window.cpp y main.cpp: zona 3, skybox, piso y reloj de animación

```
    //--- planeta 3: Verdania (Abraham) ---
    //N lleva a la superficie de Verdania, que es la zona 3; C regresa al espacio igual que en los otros planetas
    if (key == GLFW_KEY_N && action == GLFW_PRESS)
    {
        theWindow->zonaPlaneta = 3;
    }

...

//--- planeta 3: Verdania (Abraham) ---
//la superficie de Verdania es otro piso aparte y lejos, igual que Xen y el planeta 2, pero hacia Z positivo
glm::vec3 posVerdania = glm::vec3(0.0f, 0.0f, 3000.0f);
Skybox skyboxVerdania;
Texture verdaniaPastoTexture;

//todas las plantas de Verdania son modelos descargados y separados en piezas por sus huesos
//los creditos estan en el LICENCIA.txt de cada una y los pivotes salen del piezas.txt de cada carpeta
//las animaciones salen del seno de anguloVerdania, que avanza cada frame igual que el de las islas de Xen;
//cada planta lo multiplica por su propia velocidad y le suma un desfase para que no se muevan todas igual
float anguloVerdania = 0.0f;
float anguloVerdaniaOffset = 2.0f;

...

    //--- planeta 3: Verdania (Abraham) ---
    //skybox de Verdania, generado por codigo con ruido; las caras van en el mismo orden: rt, lf, dn, up, bk, ft
    std::vector<std::string> skyboxVerdaniaFaces;
    skyboxVerdaniaFaces.push_back("Textures/Skybox/verdania_rt.png");
    skyboxVerdaniaFaces.push_back("Textures/Skybox/verdania_lf.png");
    skyboxVerdaniaFaces.push_back("Textures/Skybox/verdania_dn.png");
    skyboxVerdaniaFaces.push_back("Textures/Skybox/verdania_up.png");
```

```
    skyboxVerdaniaFaces.push_back("Textures/Skybox/verdania_bk.png");
    skyboxVerdaniaFaces.push_back("Textures/Skybox/verdania_ft.png");
    skyboxVerdania = Skybox(skyboxVerdaniaFaces);

    verdaniaPastoTexture = Texture("Textures/verdania_pasto.png");
    verdaniaPastoTexture.LoadTextureA();

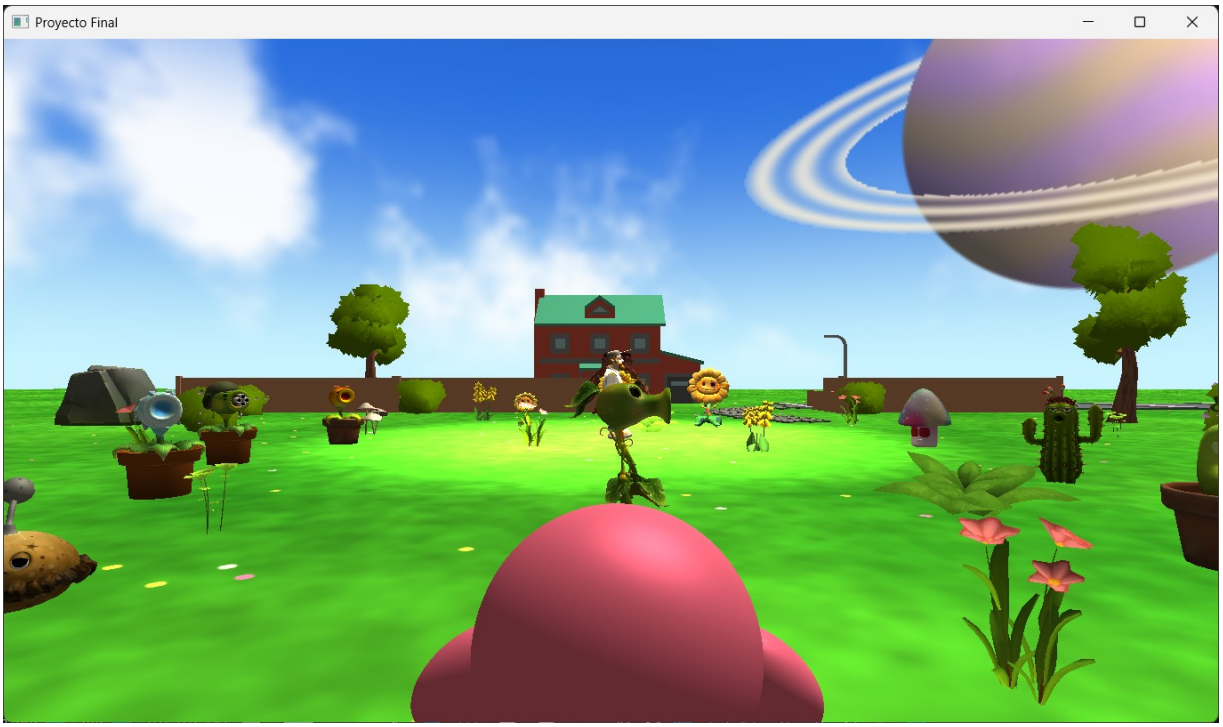
...

        //--- planeta 3: Verdania (Abraham) ---
        //el angulo comun de las animaciones avanza cada frame y se regresa a 0 al dar la vuelta
        anguloVerdania += anguloVerdaniaOffset * deltaTime;
        if (anguloVerdania > 360.0f)
            anguloVerdania -= 360.0f;

        Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);

        //piso de pasto, del mismo tamano que el de Xen (200 por 200)
        model = glm::mat4(1.0);
        model = glm::translate(model, posVerdania);
        model = glm::scale(model, glm::vec3(10.0f, 1.0f, 10.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        verdaniaPastoTexture.UseTexture();
        meshListModel[0]->RenderMeshModel();
```

Capturas de pruebas de funcionamiento



Captura 7: La superficie de Verdania recién llegando (tecla V): el piso de pasto con el skybox de Verdania al fondo.

Entrada 5 (24 de octubre): NPCs caminando (Peashooter y Crazy Dave)

Commits: f75d4bd

Qué me propongo hacer: Poner al Peashooter y a Crazy Dave en Verdania y hacer que caminen.

Horas trabajadas: 2 horas

• **Carga por piezas.** Cada planta tiene sus arreglos de piezas, padres, pivotes y rot, como la Carnívora. Las rutas se arman en un for con el nombre de cada pieza: "Models/Peashooter/peashooter_" + pieza + ".obj".

• **Peashooter en círculo.** Camina en círculo como el headcrab de Xen: su ángulo avanza con deltaTime y su posición es (r·sen θ, 0, r·cos θ). Se gira θ + 90° para que mire hacia donde avanza. Sus cuatro hojas-pata se mueven cruzadas con pasoPea = sen(anguloVerdania·3): la delantera izquierda junto con la trasera derecha, como camina un animal de cuatro patas.

• **Crazy Dave de ida y vuelta.** Va y viene en línea recta en X entre −4 y 4 con un bool que cambia al llegar al límite, como los paneles del satélite de la práctica 9. Las piernas se mueven contrarias con pasoDave = sen(anguloVerdania·4). Las pantorrillas y los pies son hijos del muslo, así que se mueven con él sin tocarlos.

• **Brazos en T.** Los modelos vienen con los brazos extendidos en T, así que se bajan con una rotación fija en Z en el hombro (70° el Peashooter y 45° Crazy Dave).

Secciones de código que modifiqué

1) main.cpp: caminata del Peashooter y de Crazy Dave

```
//el peashooter camina en circulo como el headcrab, pero sin tecla, siempre esta caminando
glm::vec3 centroPeashooter = glm::vec3(0.0f, 0.0f, -2.0f); //el centro del circulo se mide desde posVerdania
float radioPeashooter = 2.5f;
float anguloPeashooter = 0.0f;
float anguloPeashooterOffset = 0.4f;

...

        //peashooter: camina en circulo; las hojas-pata van y vienen cruzadas (la delantera izquierda con la trasera
derecha)
        anguloPeashooter += anguloPeashooterOffset * deltaTime;
        if (anguloPeashooter > 360.0f)
            anguloPeashooter -= 360.0f;
        float pasoPea = sin(anguloVerdania * 3.0f * toRadians);
        rotPeashooter[8].x = 25.0f * pasoPea;   //hoja_del_izq
        rotPeashooter[11].x = 25.0f * pasoPea;  //hoja_tras_der
        rotPeashooter[9].x = -25.0f * pasoPea;  //hoja_del_der
        rotPeashooter[10].x = -25.0f * pasoPea; //hoja_tras_izq
        //los brazos vienen extendidos en T en el modelo; se bajan 70 grados en Z y se balancean en X al caminar
        rotPeashooter[6].z = -70.0f;
        rotPeashooter[7].z = 70.0f;
        rotPeashooter[6].x = -20.0f * pasoPea;
        rotPeashooter[7].x = 20.0f * pasoPea;
        //la cabeza rebota dos veces por cada paso y el tallo se mece un poco
        rotPeashooter[3].x = 4.0f * sin(anguloVerdania * 6.0f * toRadians);
        rotPeashooter[1].z = 3.0f * pasoPea;
        model = glm::mat4(1.0);
```

```
        model = glm::translate(model, posVerdania + centroPeashooter +
            glm::vec3(radioPeashooter * sin(anguloPeashooter * toRadians), 0.0f, radioPeashooter *
cos(anguloPeashooter * toRadians)));
        model = glm::rotate(model, (anguloPeashooter + 90.0f) * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
        DibujaJerarquia(Peashooter_M, padresPeashooter, pivotesPeashooter, rotPeashooter, NUM_PIEZAS_PEASHOOTER,
model, uniformModel);

        //crazy dave: camina de un lado a otro; al llegar al limite se voltea y regresa
        if (daveAdelante)
        {
            posDave += posDaveOffset * deltaTime;
            if (posDave > limiteDave)
                daveAdelante = false;
        }
        else
        {
            posDave -= posDaveOffset * deltaTime;
            if (posDave < -limiteDave)
                daveAdelante = true;
        }
        float pasoDave = sin(anguloVerdania * 4.0f * toRadians);
        //piernas contrarias, y cada brazo va al reves que su pierna, como cuando camina una persona
        rotCrazyDave[11].x = 25.0f * pasoDave;  //muslo_der
        rotCrazyDave[14].x = -25.0f * pasoDave; //muslo_izq
        //los brazos vienen abiertos; se bajan 45 grados en Z y se balancean en X
        rotCrazyDave[5].z = 45.0f;
        rotCrazyDave[8].z = -45.0f;
        rotCrazyDave[5].x = -20.0f * pasoDave;
        rotCrazyDave[8].x = 20.0f * pasoDave;
        //la olla de la cabeza se tambalea
        rotCrazyDave[4].z = 6.0f * sin(anguloVerdania * 8.0f * toRadians);
        model = glm::mat4(1.0);
        model = glm::translate(model, posVerdania + glm::vec3(posDave, 0.0f, -8.0f));
        //el modelo mira hacia +Z; con 90 grados mira hacia +X y con -90 hacia -X
        if (daveAdelante)
            model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
        else
            model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
        DibujaJerarquia(CrazyDave_M, padresCrazyDave, pivotesCrazyDave, rotCrazyDave, NUM_PIEZAS_CRAZYDAVE, model,
uniformModel);
```

Capturas de pruebas de funcionamiento



Captura 8: El Peashooter caminando en círculo en Verdania (con las animaciones prendidas con F), de preferencia con las hojas-pata en medio del paso.



Captura 9: Crazy Dave caminando de un lado a otro con su olla en la cabeza.

Entrada 6 (27 de octubre): Verdania vista desde el espacio y cambio de planeta con Z y X

Commits: bbc7e77, ca7e2f7

Qué me propongo hacer: Que Verdania se vea desde el espacio como la cabeza de un Peashooter y que se pueda llegar a ella con las teclas Z y X.

Horas trabajadas: 2 horas

• **Planeta con forma de cabeza de Peashooter.** En el espacio, Verdania es la cabeza del Peashooter en grande, junto a Pop Star (el planeta de Kirby). Gira despacio sobre Y igual que Xen. Como el origen del modelo está en los pies, primero se traslada el centro de la cabeza al origen (translate(−centroCabezaPea)) y después se escala por 130 y se gira; así gira sobre sí misma y no alrededor de los pies.

• **Planeta liso y con textura.** La primera versión usaba las piezas del NPC, pero tenía picos en el cuello y las hojas y las cejas sobresalían. Se hizo peashooter_planeta.obj: la cabeza sin cuello, redondeada por abajo, sin cejas ni hojas, y con una textura de planeta aplicada con coordenadas esféricas para que se viera como un planeta de verdad.

• **Z y X recorren los planetas.** Antes cada planeta tenía su propia tecla. Se cambió para que X sume 1 a zonaPlaneta y Z le reste 1, y al pasar del último se regrese al primero (1 Xen, 2 Pop Star, 3 Verdania). C sigue regresando al espacio.

Secciones de código que modifiqué

1) main.cpp y Window.cpp: planeta en el espacio y teclas Z/X

```
//Verdania vista desde el espacio: es la cabeza del peashooter (con sus dos hojas de pelo) en grande,
//se reusan las piezas que ya se cargaron para el NPC, asi que no se carga ningun modelo nuevo
glm::vec3 posPlanetaVerdania = glm::vec3(-450.0f, 70.0f, -330.0f);
//centro de la cabeza en las coordenadas del modelo; ahi se gira y se escala para que no gire alrededor de los pies
glm::vec3 centroCabezaPea = glm::vec3(0.0f, 1.15f, 0.05f);
float giroPlanetaVerdania = 0.0f;
float giroPlanetaVerdaniaOffset = 0.05f;

...

        //planeta Verdania en el espacio: gira despacio sobre Y como Xen; mide como 0.73 de alto, con escala 130
queda de unas 95 unidades
        giroPlanetaVerdania += giroPlanetaVerdaniaOffset * deltaTime;
        if (giroPlanetaVerdania > 360.0f)
            giroPlanetaVerdania -= 360.0f;
        model = glm::mat4(1.0);
        model = glm::translate(model, posPlanetaVerdania);
        model = glm::rotate(model, giroPlanetaVerdania * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
        model = glm::scale(model, glm::vec3(130.0f, 130.0f, 130.0f));
        model = glm::translate(model, -centroCabezaPea);
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        Peashooter_M[3].RenderModel(); //cabeza
        Peashooter_M[4].RenderModel(); //pelo_izq
        Peashooter_M[5].RenderModel(); //pelo_der

...

-//Verdania vista desde el espacio: es la cabeza del peashooter (con sus dos hojas de pelo) en grande,
-//se reusan las piezas que ya se cargaron para el NPC, asi que no se carga ningun modelo nuevo
```

```
+//Verdania vista desde el espacio: es la cabeza del peashooter en grande, lisa y con textura de planeta
+//peashooter_planeta.obj es la cabeza sin cuello, sin cejas y redonda por abajo; su textura es la de Verdania
+Model PlanetaVerdania_M;

...

+    PlanetaVerdania_M = Model();
+    PlanetaVerdania_M.LoadModel("Models/Peashooter/peashooter_planeta.obj");
+

...

-        Peashooter_M[3].RenderModel(); //cabeza
-        Peashooter_M[4].RenderModel(); //pelo_izq
-        Peashooter_M[5].RenderModel(); //pelo_der
+        PlanetaVerdania_M.RenderModel();

...

    //Z y X rotan entre los planetas: X va al siguiente y Z al anterior (1 Xen, 2 planeta 2, 3 Verdania)
    //al pasar del ultimo se regresa al primero y al reves; C regresa al espacio
    if (key == GLFW_KEY_X && action == GLFW_PRESS)

...

        theWindow->zonaPlaneta += 1;
        if (theWindow->zonaPlaneta > 3)
            theWindow->zonaPlaneta = 1;

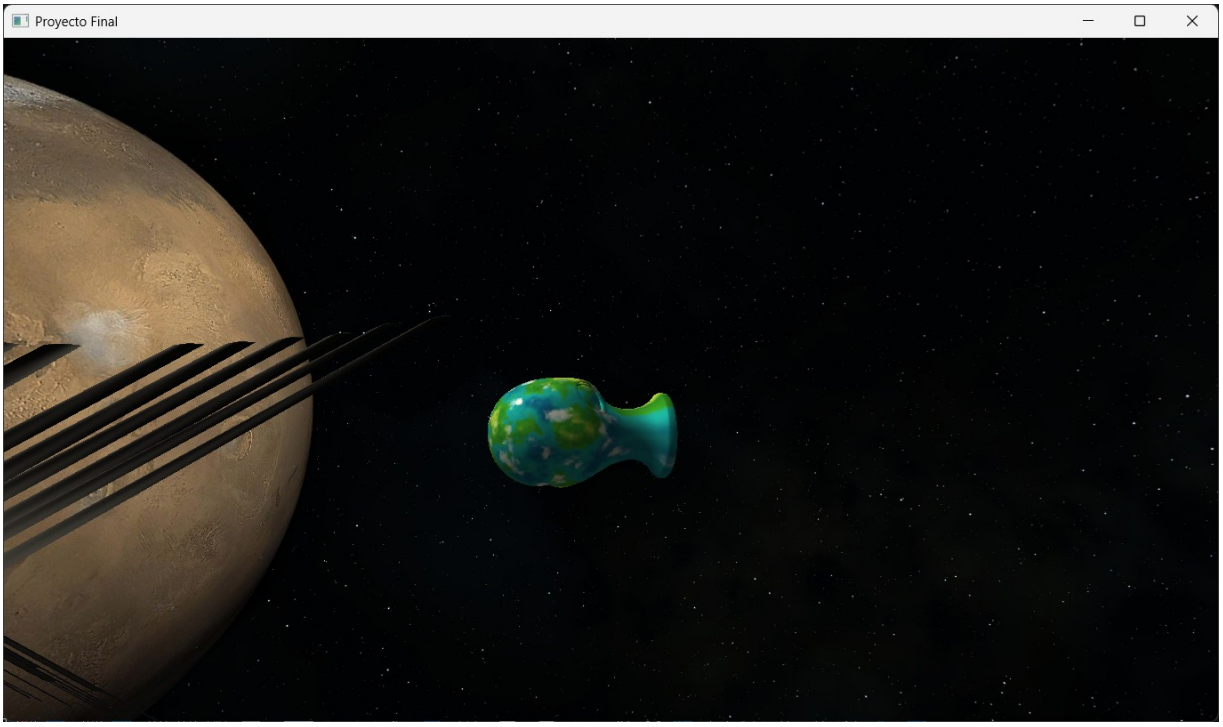
...

    if (key == GLFW_KEY_Z && action == GLFW_PRESS)

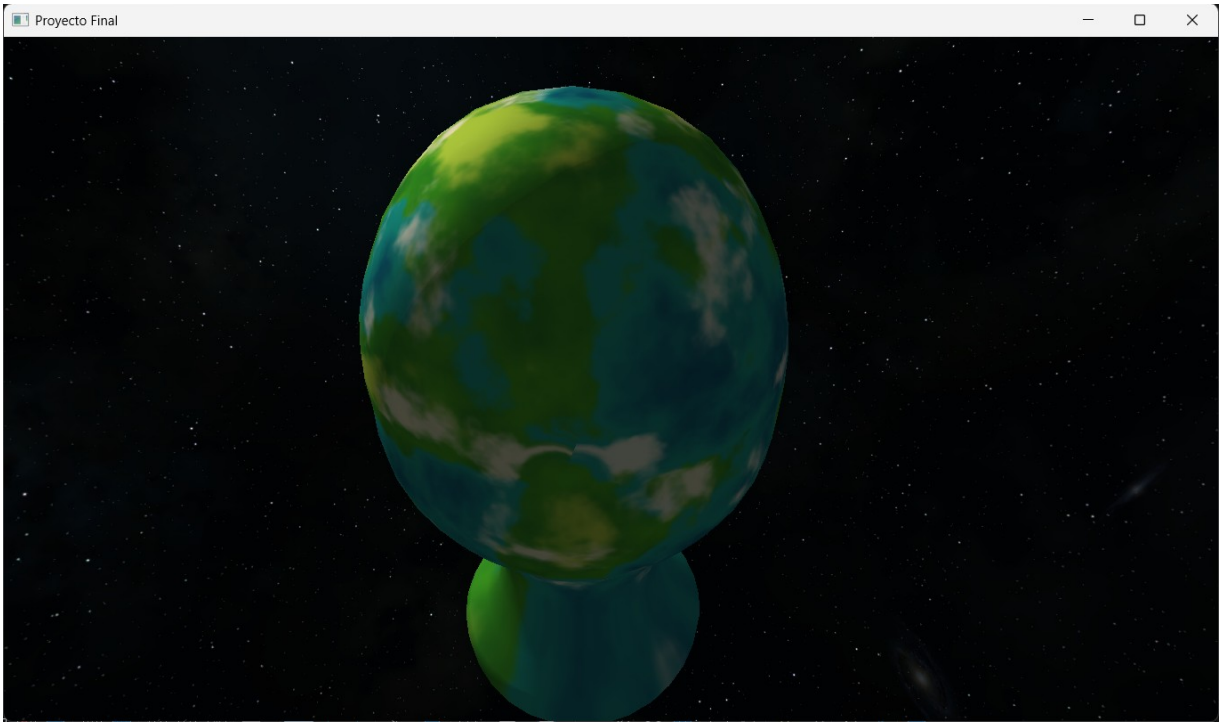
...

        theWindow->zonaPlaneta -= 1;
        if (theWindow->zonaPlaneta < 1)
            theWindow->zonaPlaneta = 3;
```

Capturas de pruebas de funcionamiento



Captura 10: Verdania en el espacio (la cabeza de Peashooter como planeta, con su textura), de preferencia con Pop Star a un lado.



Captura 11: Acercamiento a la parte de abajo del planeta, donde se ve que ya no tiene picos ni cuello.

Entrada 7 (27 de octubre): tecla para las animaciones, demás NPCs y plantas de adorno

Commits: 8ca50e5

Qué me propongo hacer: Prender y apagar las animaciones con el teclado, quitar los saltos al terminar cada vuelta y agregar los demás NPCs y plantas de adorno.

Horas trabajadas: 2 horas

• **Animación básica con teclado.** Se agregó a Window la variable animaVerdania con su getter, su valor inicial (false) en los dos constructores y una tecla que la prende o la apaga, igual que la caminata del headcrab. En main.cpp, el reloj anguloVerdania, el círculo del Peashooter, la caminata de Crazy Dave y el giro de los cañones sólo avanzan dentro de if (getanimaVerdania()). Si se apaga, las plantas se quedan en su última pose. (Después, al integrar main, el equipo reorganizó las teclas y quedó F para las animaciones y V para ir a Verdania.)

• **Movimiento sin brincos.** Cada pieza usa amplitud · sen(anguloVerdania · N + desfase). N siempre es entero, para que el seno valga lo mismo en 360° que en 0° y la animación no brinque cuando el reloj regresa a 0. Los desfases de 120° y 240° hacen que plantas iguales no se muevan al mismo tiempo.

• **Vuelta de Crazy Dave.** Antes giraba 180° de golpe al llegar al límite. Se agregó rumboDave, que se acerca a 90° o a −90° de poco en poco (rumboDaveOffset · deltaTime), para que la vuelta se vea natural.

• **Nuevos NPCs:** Ice Pea, Gatling Pea y Fire Pea en maceta (vigilan volteando la cabeza) y el Cactus (sube y baja los brazos). Los cañones del Gatling Pea son una pieza aparte, hija de la boca, que gira sin parar en su eje Z.

• **Plantas de adorno:** tres girasoles, Hypno-shroom, Laser Bean, Potato Mine, Wall-nut y Torchwood, cada uno con una animación sencilla. La Wall-nut es una sola pieza, así que se balancea todo el modelo.

Secciones de código que modifiqué

1) Tecla, rumbo de Crazy Dave, Gatling Pea y bloque de animación

```
    //--- planeta 3: Verdania (Abraham) ---
    //V prende o apaga las animaciones de las plantas de Verdania, como la H con el headcrab
    if (key == GLFW_KEY_V && action == GLFW_PRESS)
    {
        theWindow->animaVerdania = !theWindow->animaVerdania;
    }

...

//hacia donde mira crazy dave, en grados alrededor de Y: 90 cuando camina hacia +X y -90 hacia -X;
//al cambiar de direccion se va girando de rumboDaveOffset en rumboDaveOffset para que la vuelta no sea de golpe
float rumboDave = 90.0f;
float rumboDaveOffset = 6.0f;

...

//gatling pea: pea con casco y canones de metralleta; los canones son una pieza aparte para que puedan girar
const int NUM_PIEZAS_GATLINGPEA = 9;
```

```
Model GatlingPea_M[NUM_PIEZAS_GATLINGPEA];
const char* piezasGatlingPea[NUM_PIEZAS_GATLINGPEA] = {
    "base", "tallo", "cabeza", "boca", "canones", "hoja_izq", "hoja_frente", "hoja_der", "hoja_atras" };
int padresGatlingPea[NUM_PIEZAS_GATLINGPEA] = {
    -1, 0, 1, 2, 3, 0, 0, 0, 0 };
glm::vec3 pivotesGatlingPea[NUM_PIEZAS_GATLINGPEA] = {
    glm::vec3(0.0000f, 0.0000f, 0.0000f),       //base
    glm::vec3(0.0000f, 0.5482f, -0.0555f),      //tallo
    glm::vec3(0.0000f, 0.9346f, -0.1253f),      //cabeza
    glm::vec3(0.0000f, 1.1496f, 0.2911f),       //boca
    glm::vec3(0.0000f, 1.1496f, 0.2911f),       //canones
    glm::vec3(0.0361f, 0.5212f, -0.0433f),      //hoja_izq
    glm::vec3(0.0005f, 0.5212f, -0.0123f),      //hoja_frente
    glm::vec3(-0.0299f, 0.5212f, -0.0441f),     //hoja_der
    glm::vec3(0.0000f, 0.5212f, -0.0783f)       //hoja_atras
};
glm::vec3 rotGatlingPea[NUM_PIEZAS_GATLINGPEA];
//angulo de los canones de la gatling pea; avanza mientras V este prendida
float giroCanones = 0.0f;
float giroCanonesOffset = 8.0f;

...

        //como funcionan las animaciones de Verdania:
        // - anguloVerdania es un "reloj" que va de 0 a 360 grados; avanza un poco cada frame (Offset * deltaTime)
        // - cada parte que se mueve usa sin(anguloVerdania * N), que va y viene suave entre -1 y 1;
        //   ese valor se multiplica por los grados maximos que se quiere mover la pieza
        // - N siempre es un numero entero (1, 2, 3...), asi el seno da exactamente lo mismo en 360 que en 0
        //   y cuando el reloj regresa a 0 la pieza sigue su movimiento sin brincar
        // - N mas grande = movimiento mas rapido; sumar grados dentro del seno (+120, +240) es un desfase,
        //   para que plantas iguales no se muevan todas al mismo tiempo
        //las animaciones solo avanzan mientras esten prendidas con la tecla V, igual que la caminata del headcrab
con H;
        //si se apagan, los angulos y posiciones ya no cambian y las plantas se quedan quietas en su ultima pose
        if (mainWindow.getanimaVerdania())
        {
            //el reloj comun avanza y al llegar a 360 se regresa a 0
            anguloVerdania += anguloVerdaniaOffset * deltaTime;
            if (anguloVerdania > 360.0f)
                anguloVerdania -= 360.0f;
            //el peashooter avanza en su circulo
            anguloPeashooter += anguloPeashooterOffset * deltaTime;
            if (anguloPeashooter > 360.0f)
                anguloPeashooter -= 360.0f;
            //crazy dave avanza hacia un lado; al pasar el limite cambia de direccion
            if (daveAdelante)
            {
                posDave += posDaveOffset * deltaTime;
                if (posDave > limiteDave)
                    daveAdelante = false;
            }
            else
            {
                posDave -= posDaveOffset * deltaTime;
                if (posDave < -limiteDave)
                    daveAdelante = true;
            }
            //crazy dave se voltea poco a poco hacia donde camina (90 grados hacia +X y -90 hacia -X),
            //en lugar de girar de golpe al llegar al limite
            if (daveAdelante && rumboDave < 90.0f)
            {
                rumboDave += rumboDaveOffset * deltaTime;
                if (rumboDave > 90.0f)
```

```
                rumboDave = 90.0f;
        }
        if (!daveAdelante && rumboDave > -90.0f)
        {
            rumboDave -= rumboDaveOffset * deltaTime;
            if (rumboDave < -90.0f)
                rumboDave = -90.0f;
        }
        //los canones de la gatling pea giran sin parar
        giroCanones += giroCanonesOffset * deltaTime;
        if (giroCanones > 360.0f)
            giroCanones -= 360.0f;
    }
```

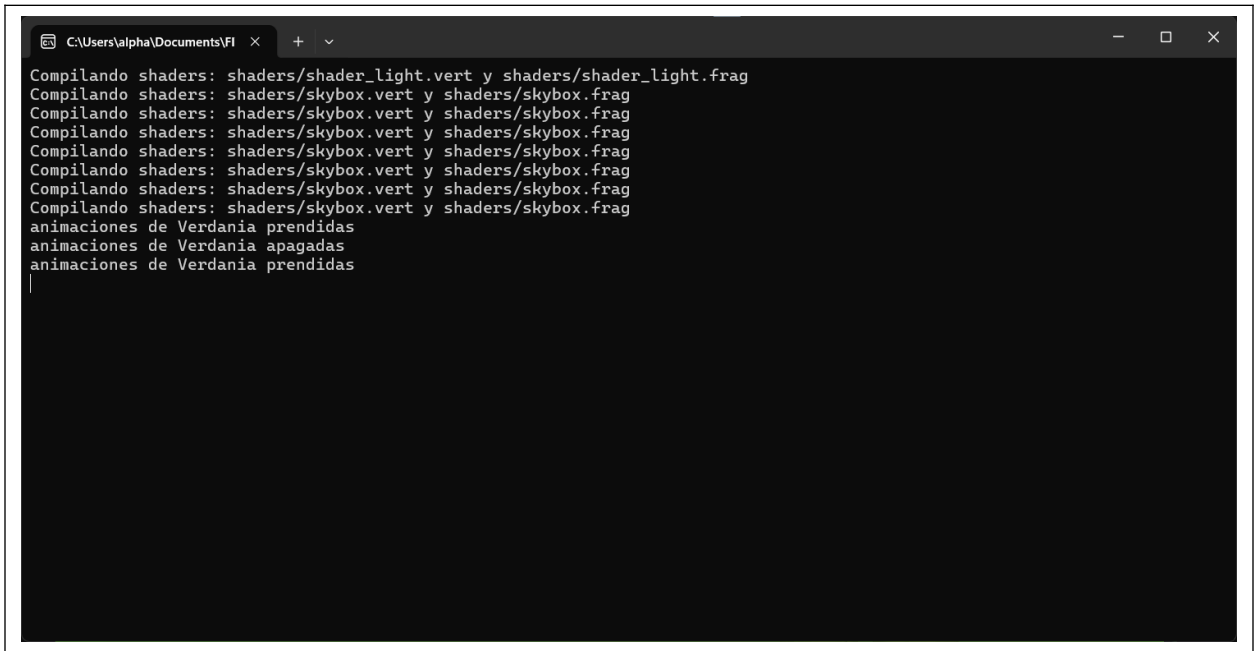
Capturas de pruebas de funcionamiento



Captura 12: Los tres peas en maceta (Ice Pea, Gatling Pea y Fire Pea) y el Cactus.



Captura 13: Las plantas de adorno: girasoles, Hypno-shroom, Laser Bean, Potato Mine, Wall-nut y Torchwood.



Captura 14: La consola mostrando el mensaje que sale al prender las animaciones con F.

Entrada 8 (27 de octubre): jardín de Crazy Dave

Commits: 63aea60

Qué me propongo hacer: Armar el jardín de Crazy Dave con su casa, la calle y el camino de piedra para que Verdania se vea más viva.

Horas trabajadas: 2.5 horas

La idea de Verdania es que Crazy Dave descubrió el planeta y armó su jardín donde ya vivían las plantas, como en el juego.

• **Modelos CC0.** La casa, la calle, las cercas y el farol son de Kenney (City Kit Suburban y Roads), y los árboles, arbustos, flores, piedras y detalles del pasto son de Quaternius (Stylized Nature MegaKit). Todos están en Models/JardinDave con su LICENCIA.txt.

• **Una calle que sigue como camino de piedra.** Los tramos de calle se dibujan en un for, girados 90° para que corran sobre X, con el final de la calle y el farol en la punta. Ahí empieza el camino de piedra, que pasa por la puerta de la cerca y entra al jardín. Cada piedra se gira distinto (i · 47°) para que no se vean repetidas.

• **Variedad con los mismos modelos.** Los árboles, arbustos y flores se alternan entre dos modelos (i % 2) y cada uno tiene su propio tamaño y giro. Los detalles del pasto se reparten con i % 3 (trébol, helecho, hongo).

• **Escala.** Los modelos de Kenney son muy chicos (una casa mide 1.2), así que se escalan por 6.5 para que la casa quede de casi 8 unidades junto a Crazy Dave, que mide 2.3.

Secciones de código que modifiqué

1) main.cpp: calle, farol, camino de piedra y cerca

```
//calle: los tramos rectos se giran 90 grados para que corran sobre X, y en la punta va el final de la calle
for (int i = 0; i < NUM_TRAMOS_CALLE; i++)
{
    model = glm::mat4(1.0);
    model = glm::translate(model, posVerdania + glm::vec3(posCalleX[i], 0.01f, posCalleZ));
    model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(escalaKenney, escalaKenney, escalaKenney));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    JardinCalle_M.RenderModel();
}
model = glm::mat4(1.0);
model = glm::translate(model, posVerdania + glm::vec3(posCalleX[0] - 6.5f, 0.01f, posCalleZ));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(escalaKenney, escalaKenney, escalaKenney));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
JardinCalleFin_M.RenderModel();

//farol junto a donde termina la calle
model = glm::mat4(1.0);
model = glm::translate(model, posVerdania + glm::vec3(posCalleX[0] - 5.0f, 0.0f, posCalleZ - 4.0f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(escalaKenney, escalaKenney, escalaKenney));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
JardinFarol_M.RenderModel();

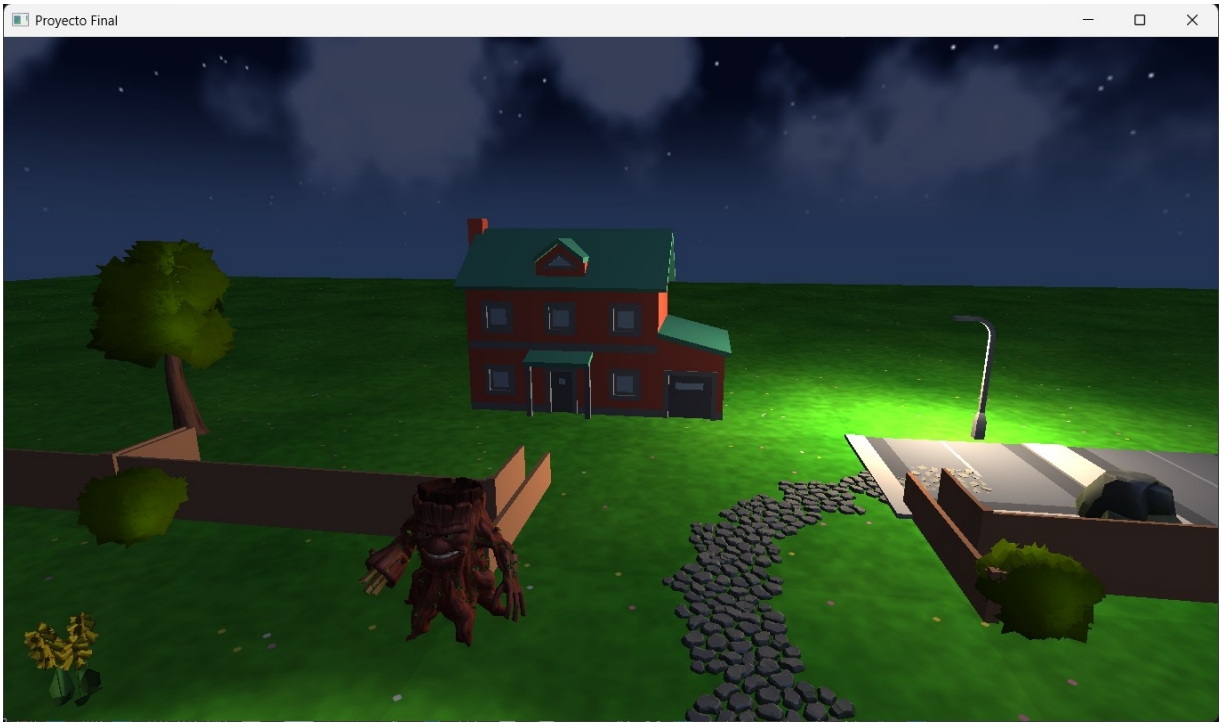
//camino de piedra; cada piedra se gira distinto para que el camino no se vea repetido
for (int i = 0; i < NUM_PIEDRAS_CAMINO; i++)
```

```
        {
            model = glm::mat4(1.0);
            model = glm::translate(model, posVerdania + posCamino[i]);
            model = glm::rotate(model, i * 47.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            JardinCaminoPiedra_M.RenderModel();
        }
```

Capturas de pruebas de funcionamiento



Captura 15: Vista general del jardín: la casa de Crazy Dave, la cerca, la calle y el camino de piedra que entra al jardín.



Captura 16: El jardín de noche (tecla N) con el farol encendido.

Entrada 9 (31 de octubre): animaciones más sencillas e integración de main

Commits: e03e91c, a7dcc9d

Qué me propongo hacer: Hacer más sencillas las animaciones de las plantas e integrar los últimos cambios de main a mi rama.

Horas trabajadas: 1.5 horas

• **Una parte animada por planta.** Había demasiadas piezas moviéndose al mismo tiempo. Se dejó sólo la parte más representativa de cada planta: el Peashooter mueve sus hojas-pata, Crazy Dave sus piernas, los peas voltean la cabeza, el Cactus mueve los brazos, los girasoles mecen la cabeza, el Hypno-shroom tambalea el sombrero, el Laser Bean balancea el cuerpo y el Torchwood abre la mandíbula. Sólo se borraron líneas; las piezas que ya no se mueven se quedan en 0° porque los arreglos rot son globales.

• **Merge con main.** main tenía avances del equipo (cámaras, avatar, luces, ciclo de día y noche) y en él se habían reescrito los comentarios de mi bloque de animaciones, así que GitHub Desktop marcó 9 conflictos en main.cpp. Se resolvieron quedándose con todo lo de main y con mis animaciones simplificadas.

Secciones de código que modifiqué

1) main.cpp: líneas que se quitaron (−) y comentarios actualizados (+)

```
-        //los brazos vienen extendidos en T en el modelo; se bajan 70 grados en Z en el hombro
-        //y se balancean en X al caminar, cada uno al reves del otro
+        //los brazos vienen extendidos en T en el modelo; solo se bajan 70 grados en Z en el hombro y se quedan
quietos

...

-        rotPeashooter[6].x = -20.0f * pasoPea;  //brazo_izq adelante y atras
-        rotPeashooter[7].x = 20.0f * pasoPea;   //brazo_der adelante y atras
-        //la cabeza rebota dos veces por cada paso (N = 6, el doble que el paso) y el tallo se mece un poco
-        rotPeashooter[3].x = 4.0f * sin(anguloVerdania * 6.0f * toRadians); //cabeza
-        rotPeashooter[1].z = 3.0f * pasoPea;                                //tallo1

...

-        //las piernas van contrarias y cada brazo va al reves que su pierna, como cuando camina una persona;
+        //solo se mueven las piernas, una al reves de la otra, como cuando camina una persona;

...

-        //los brazos vienen abiertos en el modelo; se bajan 45 grados en Z y se balancean en X
+        //los brazos vienen abiertos en el modelo; solo se bajan 45 grados en Z y se quedan quietos

...

-        rotCrazyDave[5].x = -20.0f * pasoDave;  //brazo_der adelante y atras
-        rotCrazyDave[8].x = 20.0f * pasoDave;   //brazo_izq adelante y atras
-        //la olla de la cabeza se tambalea rapido (N = 8)
-        rotCrazyDave[4].z = 6.0f * sin(anguloVerdania * 8.0f * toRadians); //olla

...

-        //los tres peas en maceta (NPCs) vigilan: voltean la cabeza de un lado a otro hasta 35 grados (en Y)
-        //y el tallo se inclina un poco hacia adelante y atras (en X); cada uno lleva un desfase distinto (0, 120 y
```

```
240)
+         //los tres peas en maceta (NPCs) vigilan: solo voltean la cabeza de un lado a otro hasta 35 grados (en Y);
+         //cada uno lleva un desfase distinto (0, 120 y 240) para que no volteen todos al mismo tiempo

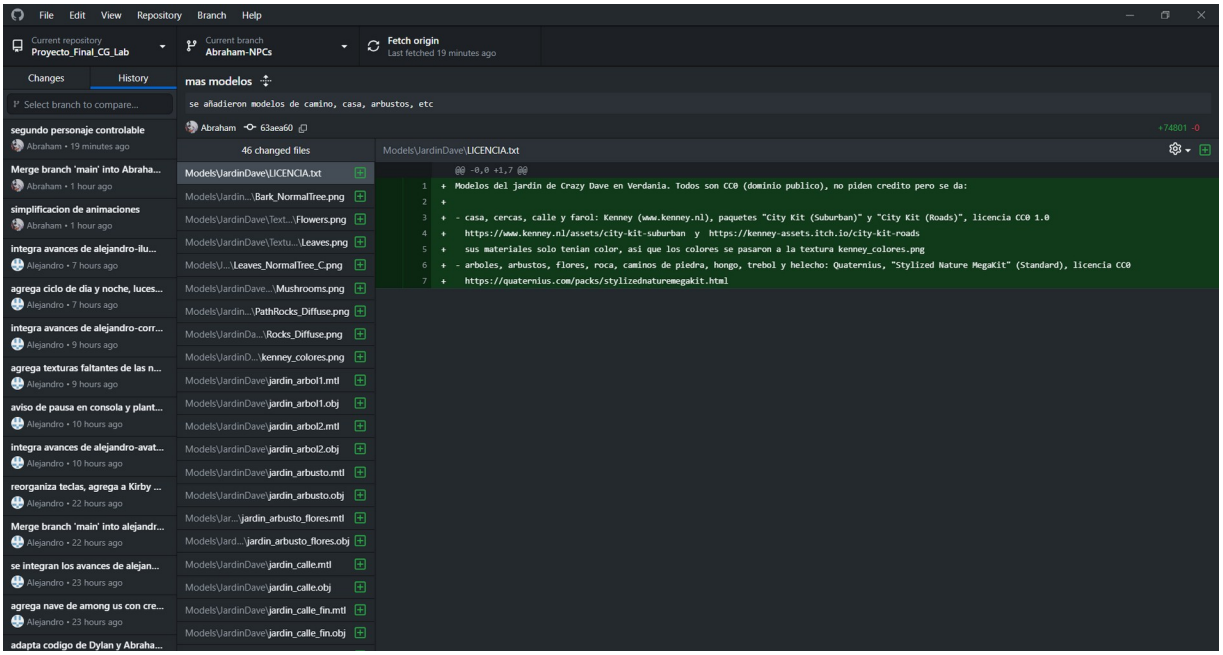
...

-         rotIcePea[1].x = 4.0f * sin(anguloVerdania * 2.0f * toRadians); //tallo

...

-         rotGatlingPea[1].x = 4.0f * sin((anguloVerdania * 2.0f + 120.0f) * toRadians); //tallo
```

Capturas de pruebas de funcionamiento



Captura 17: GitHub Desktop en la pestaña History de la rama Abraham-NPCs, mostrando los commits «simplificacion de animaciones» y «Merge branch 'main' into Abraham-NPCs».

Entrada 10 (31 de octubre): segundo personaje controlable

Commits: 547016d

Qué me propongo hacer: Poder cambiar entre Kirby y la Carnívora como personaje controlable.

Horas trabajadas: 1.5 horas

El avatar del proyecto es Kirby, que se controla con la cámara 2 en cualquier planeta. Se agregó que con la tecla T se pueda cambiar a la Carnívora, que es mi avatar.

• **Window.** Se agregó int personajeAvatar (0 es Kirby y 1 la Carnívora) con su getter, su valor inicial en los dos constructores y la tecla T, que le suma 1 y regresa a 0 al pasar del último personaje. Es int y no bool para que después se puedan agregar más personajes. En la consola se avisa cuál quedó.

• **Rotaciones propias.** La Carnívora avatar usa las mismas piezas, padres y pivotes que la de la sección de personajes, pero su propio arreglo rotAvatarCarnivora, para no mover a la otra al caminar.

• **Caminata.** Las hojas-pata usan el mismo pasoAvatar que mueve los pies de Kirby, cruzadas como las del Peashooter.

• **Dibujo.** La posición, el giro, la escala y la cámara son los mismos para los dos personajes; un if sólo decide cuál modelo se dibuja.

En este mismo commit quedó que el Gatling Pea sólo gire sus cañones y que el Laser Bean ya no mueva su antena por separado.

Secciones de código que modifiqué

1) Window.h, Window.cpp y main.cpp: tecla T y Carnívora como avatar

```
    //T cambia el personaje que se controla con la camara 2: 0 es Kirby y 1 la carnivora;
    //al pasar del ultimo personaje se regresa al primero
    if (key == GLFW_KEY_T && action == GLFW_PRESS)
    {
        theWindow->personajeAvatar += 1;
        if (theWindow->personajeAvatar > 1)
            theWindow->personajeAvatar = 0;
        //se avisa en la consola cual personaje quedo
        if (theWindow->personajeAvatar == 0)
            printf("avatar: Kirby\n");
        else
            printf("avatar: carnivora\n");
    }

...

    int getpersonajeAvatar() { return personajeAvatar; }

...

    int personajeAvatar; //personaje que se controla con la camara 2, vale 0 si es Kirby y 1 si es la carnivora, y
lo cambia la T

...
```

```
//la carnivora tambien puede ser el avatar (la T cambia de personaje); usa las mismas piezas, padres y pivotes
//que la de la seccion de personajes, pero sus propias rotaciones para no mover a la otra carnivora al caminar
glm::vec3 rotAvatarCarnivora[NUM_PIEZAS_CARNIVORA];

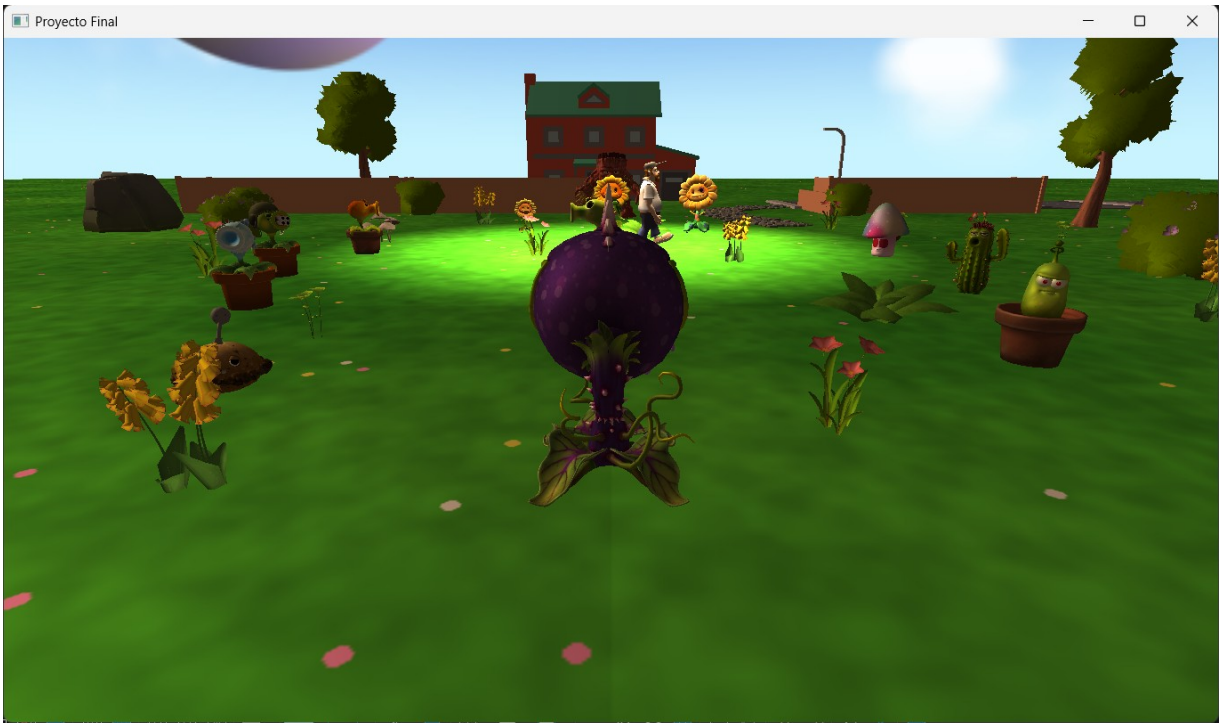
...

+       //si el avatar es la carnivora, sus hojas-pata usan el mismo pasoAvatar y se mueven cruzadas como las del
peashooter:
+       //la delantera izquierda junto con la trasera derecha, y la delantera derecha junto con la trasera
izquierda
+       rotAvatarCarnivora[8].x = pasoAvatar;   //hoja_del_izq
+       rotAvatarCarnivora[9].x = pasoAvatar;   //hoja_tras_der
+       rotAvatarCarnivora[7].x = -pasoAvatar;  //hoja_del_der
+       rotAvatarCarnivora[10].x = -pasoAvatar; //hoja_tras_izq

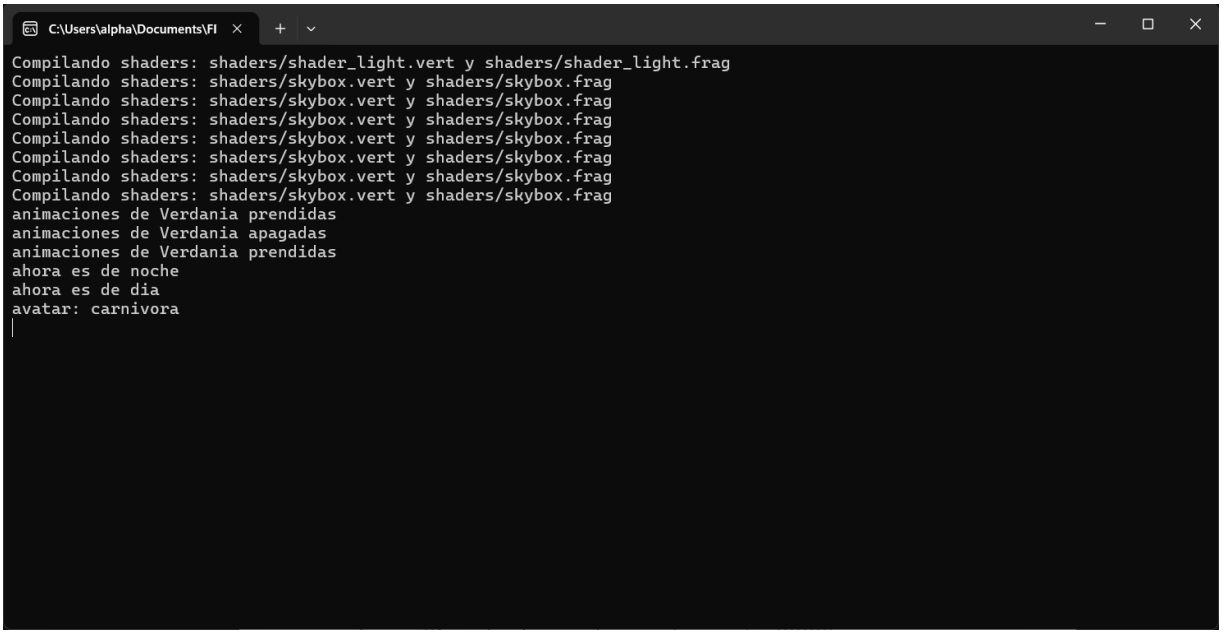
...

-           DibujaJerarquia(Kirby_M, padresKirby, pivotesAvatar, rotAvatar, NUM_PIEZAS_KIRBY, model, uniformModel);
+           //los dos personajes se colocan, giran y escalan igual; la T solo cambia cual modelo se dibuja
+           if (mainWindow.getpersonajeAvatar() == 0)
+               DibujaJerarquia(Kirby_M, padresKirby, pivotesAvatar, rotAvatar, NUM_PIEZAS_KIRBY, model,
uniformModel);
+           else
+               DibujaJerarquia(Carnivora_M, padresCarnivora, pivotesCarnivora, rotAvatarCarnivora,
NUM_PIEZAS_CARNIVORA, model, uniformModel);
```

Capturas de pruebas de funcionamiento



Captura 18: La Carnívora como avatar en la cámara 2 (teclas 2 y luego T), caminando en Verdania.



Captura 19: La consola mostrando «avatar: carnivora» después de presionar T.

Problemas encontrados durante el desarrollo

1.- Las texturas de los personajes no cargaban: Model.cpp sólo buscaba en Textures/ y el equipo había acordado poner las texturas de cada modelo en su propia carpeta. Se guardó la carpeta del .obj al cargarlo y se busca primero ahí; si no está, en Textures/ como antes (entrada 2).

2.- Error al compilar el planeta: Se usaba PlanetaVerdania_M.RenderModel() sin haber declarado Model PlanetaVerdania_M; con las demás variables globales. Se agregó la declaración arriba, junto a posPlanetaVerdania.

3.- Llave faltante en Window.cpp: Al cambiar las teclas Z y X por la rotación entre planetas se borró por error la llave } que cerraba el bloque de la Z, y todo lo que seguía quedaba dentro de ese if. Se volvió a poner la llave.

4.- Corte brusco al terminar cada vuelta de las animaciones: Algunas piezas usaban velocidades no enteras (0.5, 0.75, 1.5…) dentro del seno, y cuando anguloVerdania regresaba de 360° a 0° el seno cambiaba de valor de golpe. Se cambiaron todas a enteros. Además, Crazy Dave giraba 180° de golpe, y se agregó rumboDave para que la vuelta fuera gradual.

5.- Picos en la parte de abajo del planeta: La cabeza del Peashooter tenía el cuello y triángulos que salían hacia abajo; quitar sólo esos triángulos no bastó. Se redondeó la parte de abajo, se cerró el hueco, se quitaron las cejas y las hojas y se le puso una textura de planeta.

6.- Nombres repetidos con el planeta 2: Variables del jardín como NUM_ARBOLES ya existían en el código del planeta 2 y no compilaba. Se renombraron con el sufijo Jardin (NUM_ARBOLES_JARDIN, etc.).

7.- Modelos del jardín blancos o con hojas negras: Los modelos de Kenney sólo tenían colores en el material y el shader del proyecto usa texturas, así que salían blancos; se pasaron

sus colores a una textura de paleta (kenney_colores.png). Las hojas de Quaternius salían negras por su canal alfa, que se rellenó, y a un arbusto con hojas rojas se le cambió la textura.

8.- Conflictos al integrar main: Hubo cambios guardados (stash) que dejaron marcas <<<<<<< en main.cpp y, en el segundo merge, comentarios de mi bloque reescritos en main. Se descartó el stash, se repitió el merge y los conflictos se resolvieron quedándose con lo de main más mis cambios.

Conclusión

a. Complejidad y explicación. Lo más laborioso fue preparar los modelos: separarlos por huesos, sacar los pivotes de cada pieza y dejar las texturas donde el proyecto las encontrara. Una vez hecho eso, agregar cada planta al código fue repetir el mismo patrón (arreglos de piezas, padres, pivotes y rot, más DibujaJerarquia). Las animaciones también siguen un patrón sencillo de explicar: un reloj que avanza con deltaTime y un seno por cada pieza que se mueve.

b. Comentarios generales. Trabajar en una rama propia permitió avanzar sin romper lo del equipo, pero los merges con main requieren cuidado, sobre todo cuando dos personas tocan las mismas líneas. También fue útil hacer commits chicos y con un solo propósito, porque así es fácil explicar por qué se hizo cada cambio.

c. Conclusión. En esta rama construí mi parte del sistema Kepler-Aula: el avatar Carnívora por piezas, el planeta Verdania en el espacio y en la superficie, los NPCs y las plantas de Plants vs. Zombies con animaciones por teclado, el jardín de Crazy Dave y la opción de cambiar de personaje. Todo se hizo reutilizando lo que ya tenía el proyecto y lo visto en clase (modelos jerárquicos, transformaciones, animación con deltaTime y funciones seno), lo que demuestra que con pocas herramientas bien entendidas se puede armar un escenario completo.

Bibliografía

Angel, E., & Shreiner, D. (2012). Interactive computer graphics: A top-down approach with shader-based OpenGL (6a ed.). Addison-Wesley.

Assimp Team. (s.f.). Open Asset Import Library documentation. https://assimp-docs.readthedocs.io/

GitHub. (s.f.). GitHub Desktop documentation. https://docs.github.com/en/desktop

Kenney. (s.f.). City Kit (Suburban) [Modelos 3D, CC0]. https://www.kenney.nl/assets/city-kit-suburban

Okruzhko, N. (s.f.). Chomper - PVZ Garden Warfare [Modelo 3D, CC-BY-4.0]. Sketchfab. https://sketchfab.com/3d-models/chomper-pvz-garden-warfare-938abc15ee7248f3a71c911b3507b59f

Okruzhko, N. (s.f.). Peashooter - PVZ Garden Warfare 2 [Modelo 3D, CC-BY-4.0]. Sketchfab. https://sketchfab.com/3d-models/peashooter-pvz-garden-warfare-2-4204531da122409abe25a9d808bff294

Okruzhko, N. (s.f.). Crazy Dave - PVZ Garden Warfare 2 [Modelo 3D, CC-BY-4.0]. Sketchfab. https://sketchfab.com/3d-models/crazy-dave-pvz-garden-warfare-2-126c7de4a4144d4499cceaa7ba0e3a39

Quaternius. (s.f.). Stylized Nature MegaKit [Modelos 3D, CC0]. https://quaternius.com/packs/stylizednaturemegakit.html

Vries, J. de. (2020). Learn OpenGL: Transformations. https://learnopengl.com/Getting-started/Transformations

Los créditos de los demás modelos de Plants vs. Zombies (Ice Pea, Gatling Pea, Fire Pea, Cactus, girasoles, Hypno-shroom, Laser Bean, Potato Mine, Wall-nut y Torchwood) están en el LICENCIA.txt de la carpeta de cada modelo.